

MindSpore Transformers 大模型系列课程

MindSpore Transformers LLM推理介绍与实践

专题一：套件介绍

《MindSpore Transformers训推Mcore架构介绍》

《MindSpore Transformers环境安装与镜像制作》

专题二：LLM训练

《MindSpore Transformers LLM预训练与微调介绍与实践》

《MindSpore Transformers LLM数据预处理介绍与实践》

《MindSpore Transformers Safetensors权重详解》

《MindSpore Transformers断点续训介绍与实践》

《MindSpore Transformers训练在线监控介绍与实践》

《MindSpore Transformers训练高可用特性介绍》

专题三：LLM推理



《MindSpore Transformers推理介绍与实践》

《MindSpore Transformers & vLLM部署与评测》

专题四：高阶开发

《MindSpore Transformers LLM推理迁移与实践》

《MindSpore Transformers & vLLM推理精度比对》

《MindSpore Transformers LLM训练迁移与实践》

《MindSpore Transformers & Megatron-LM训练精度比对》

《MindSpore Transformers训练性能调优》

《MindSpore Transformers推理性能调优》

《MindSpore Transformers DeepSeek-R1蒸馏实践》

目录

1. LLM大语言模型发展介绍
2. MindSpore Transformers推理全流程能力一览
3. MindSpore Transformers量化推理能力一览
4. MindSpore Transformers推理任务快速上手

1. LLM大语言模型发展介绍

感知理解世界，专用领域

生成创造世界，通用领域

替代低端重复性工作

可以物体识别和分析，可部分替代人眼/耳，主要用于物体识别和发现

替代较高端脑力劳动工作

会思考会学习，主要用于理解和生成，Transformer架构为主

AI技术
里程碑

控制论

NLP

CV

深度学习

GAN

AI4S

Stable Diffusion

ChatGPT

GPT-4o

GPT-O1

*图像生成
~4000 高性能AI卡

*文本生成
~1万+高性能AI卡

*多模态交互

*逻辑推理/慢思考



DeepSeek-R1

AI产业
“三叠浪”

**第二次浪潮：
专家系统+机器学习**
(1976~2006：
人工智能步入产业化)

第三次浪潮：深度学习
(2006~2022：
泛化能力提升，人工智能爆发)

第四次浪潮：智能内容生成
(2022~?：生成内容，深度思考)

DeepSeek-V3/R1

- 6710亿参数MoE大模型
- 强化学习驱动的创新训练方法
- 有效平衡低成本与高性能
- 高效推理与强大数学能力
- ...

**第一次浪潮：
符号主义+逻辑推理**
(1956~1976：
人工智能诞生)

专家
系统

视觉识别
广告推荐

ChatGPT：

- 通过大算力+算法创新孵化催生
- 5天用户破百万，2个月活破亿
- 通过谷歌L3工程师编程考试
- 比尔盖茨：历史意义重大，不亚于PC或互联网诞生

AI重点事件
里程碑

1956年
达特茅斯
会议

1997年
深蓝机器人
战胜人类

2015年
视觉识别超过人类
ResNet-50

2020年
蛋白质结构预测
Alpha Fold 2

2022年
ChatGPT

2023年
GPT-4

2024年
GPT-4o/O1
DeepSeek-V3

**2025年
DeepSeek-R1**

**持续推出
新的语言模型**

2. MindSpore Transformers推理全流程能力一览



MindSpore Transformers大模型推理技术架构全新升级

离线推理 & 在线部署

Hugging Face权重加载

配置化模型开发

注册机制一键部署

Models

Hugging Face
模型配置



Transformer Layer
API接口



DeepSeek-V3

Qwen3/Qwen3-MoE

Llama3.1/3/2

GLM4/4.5

...

Hugging Face
配置原生加载：0 Day迁移

Megaton-LM API对齐：Cell级精度比对

模块化模型接口（Module Spec模块化组装）

GPTModel

TransformerBlock

TransformerLayer

MultiTokenPrediction

MoELayer

MLP

MultiLatentAttention

FlashAttention

ColumnParallelLinear

RowParallelLinear

VocabParallelEmbedding

...

多维混合并行

模型并行(TP)

数据并行(DP)

专家并行(EP)

流水线并行(PP)

序列并行(SP/CP)

服务化部署 | 榜单评测

vLLM-MindSpore Plugin/
SGLang-MindSpore Plugin

- 接口兼容
- 组件解耦
- 最小化侵入式修改

AISBench基准工具/
OpenCompass榜单评测

- 支持20+主流数据集
- 服务化数据集精度验证
- 延迟和吞吐性能压测

模型快速迁移 | 精度比对

Hugging Face配置原生加载

MCore接口精度对齐vLLM

Hugging Face Tokenizer
无缝对接

快速权重转换

Hugging Face权重在线加载

Cell级dump工具

量化推理 | 高性能算子

- 支持金箍棒量化校准
- 支持HF/ModelSlim量化权重推理
- A16W8/A8W8/A8W4/C8...

- 对接昇腾ATB和ACLNN算子库
- 自研融合算子优化
- 自定义算子接入（能力构建中）

模型开箱即用

Hugging Face主流模型0day迁移

易用/高性能推理

MindSpore Transformers推理全流程

任务前准备

修改推理配置

权重加载

执行推理任务

- 确定指定规格模型
- 确定模型来源

权重配置
并行配置
推理模型配置
文本生成配置

支持在线加载

- 训转推权重
- HF权重

msrun快速启动

单机单卡 | 单机多卡

任务前准备

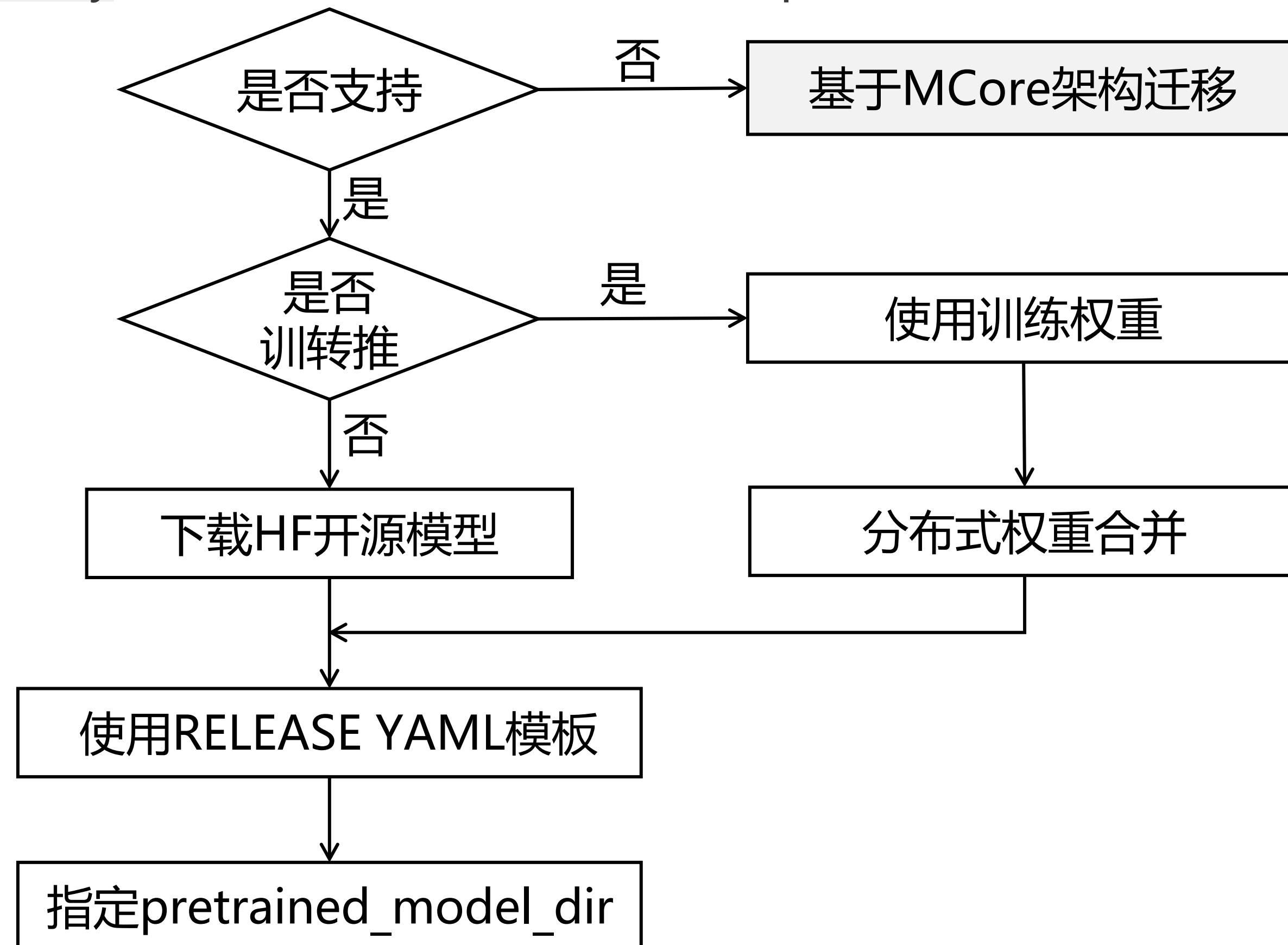
修改推理配置

权重加载

执行推理任务

选择模型

- Released (发布级)：通过测试团队验收，评测精度与标杆满足对齐标准；
- Validated (验证级)：通过开发团队自验证，评测精度与标杆满足对齐标准；
- Preliminary (初步级)：通过开发者初步自验证，功能完整可试用，推理输出符合逻辑但精度未严格验证；
- Untested (未测试级)：功能可用但未经系统测试，精度未验证，支持用户自定义开发使能；
- Community (社区级)：社区贡献的 MindSpore 原生模型，由社区开发维护。



```
git clone https://huggingface.co/XXX/XXX
.
├── config.json
├── generation_config.json
├── merges.txt
├── model-xxx-of-xxx.safetensors
├── README.md
├── tokenizer_config.json
├── tokenizer.json
└── vocab.json
```

HF开源模型：生态支持直接使用

```
python toolkit/safetensors/unified_safetensors.py
--src_strategy_dirs output/strategy \
--mindsporeckpt_dir output/checkpoint \
--output_dir /path/to/unified_trainckpt \
--file_suffix "1000_1" \
--filter_out_param_prefix "adam_" \
--has_redundancy False
```

训转推模型：合并分布式权重
完整权重可直接用于推理任务

```
pretrained_model_dir: "/path/to/Qwen3-32B"
model:
  model_config:
    params_dtype: "float32"
    compute_dtype: "bfloat16"
    layernorm_compute_dtype: "float32"
    softmax_compute_dtype: "float32"
  ...
```

修改模型文件夹

指定模型路径：将pretrained_model_dir配置成模型文件夹
文件夹为模型权重文件、tokenizer文件和相关json文件

```
pretrained_model_dir: "/path/to/Qwen3-8B"
model:
  model_config:
    params_dtype: "float32"
    compute_dtype: "bfloat16"
    layernorm_compute_dtype: "float32"
    softmax_compute_dtype: "float32"
  ...
```


任务前准备

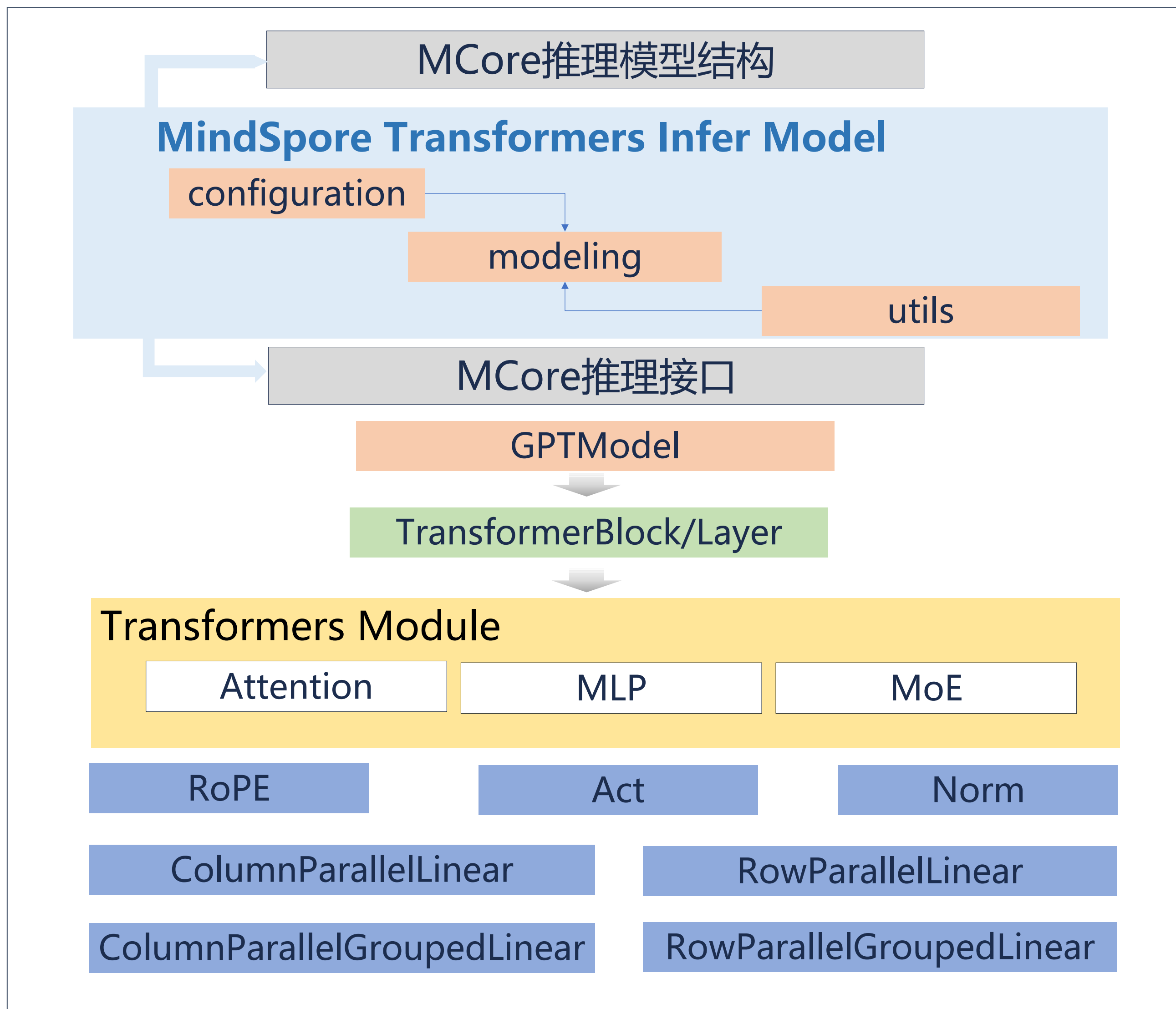
修改推理配置

权重加载

执行推理任务

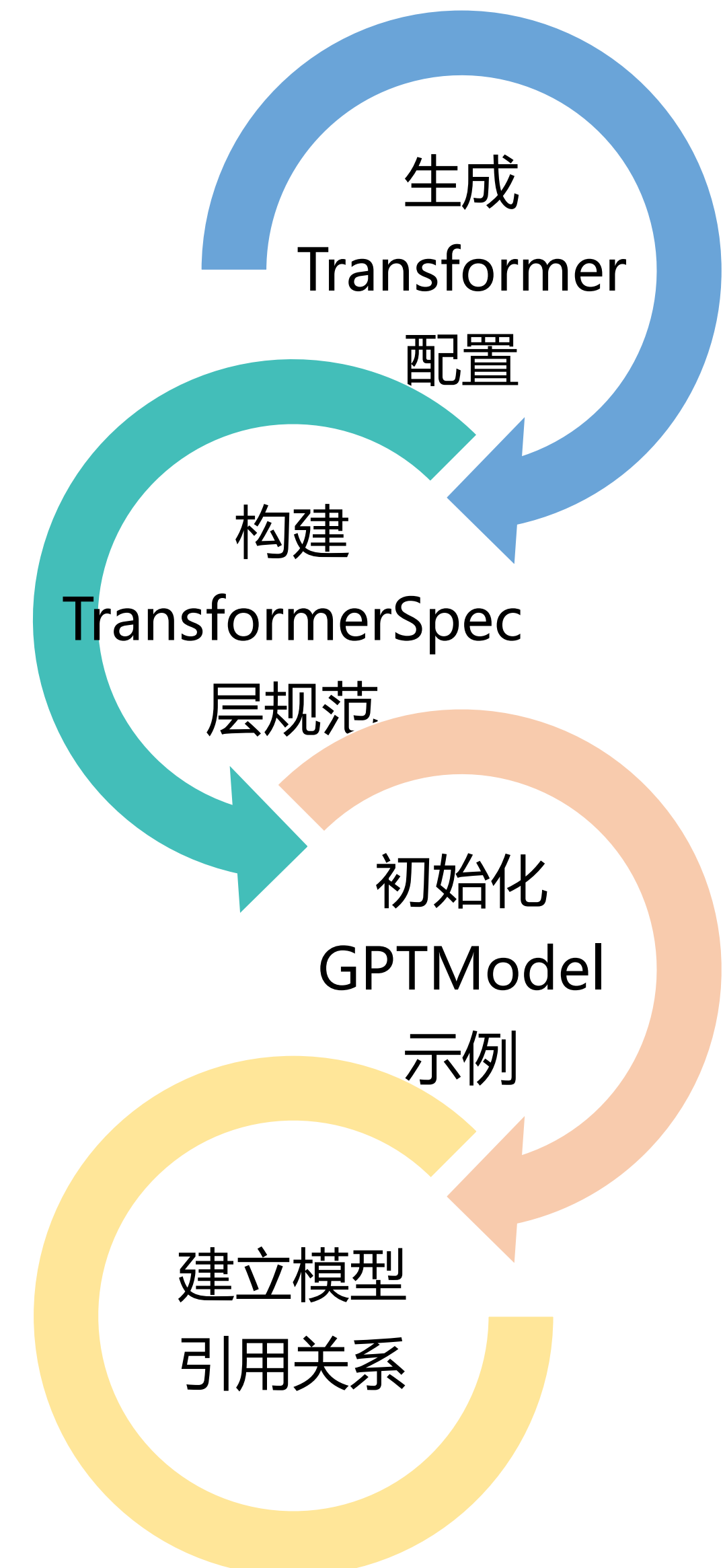
MCORE模型迁移

MCORE架构提供了从底层如ColumnParallelLinear到上层如TransformerLayer的功能封装模块，并支持通过ModuleSpec类进行组合。

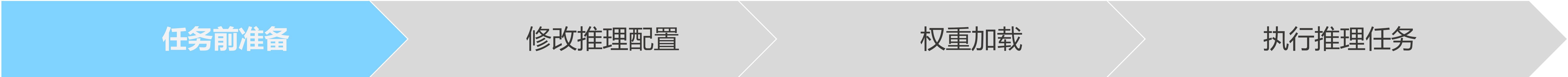


```
# 推理MCORE模型需继承InferModelMixin和对应的PreTrainedModel
class InferenceMyModelForCausalLM(MyModelPreTrainedModel,
InferModelMixin):

    def __init__(self, config):
        super().__init__(config, auto_prefix=False)
        self.config = config
        # step1: 由模型config转换生成transformer config
        config: TransformerConfig =
        convert_to_transformer_config(self.config)
        # step2: 构建TransformerSpec层规范
        transformer_layer_spec =
        get_gpt_decoder_block_spec(transformer_config)
        # step3: 初始化GPTModel实例
        gpt_model = GPTModel(
            config=config,
            transformer_layer_spec=transformer_layer_spec,
            vocab_size=self.vocab_size,
            max_sequence_length=self.max_position_embeddings,
            position_embedding_type=config.position_embedding_type,
            rotary_base=self.config.rope_theta,
            post_process=self.config.post_process,
            model_comm_pgs=model_comm_pgs
        )
        # step4: 建立模型引用关系
        self.model = gpt_model
```

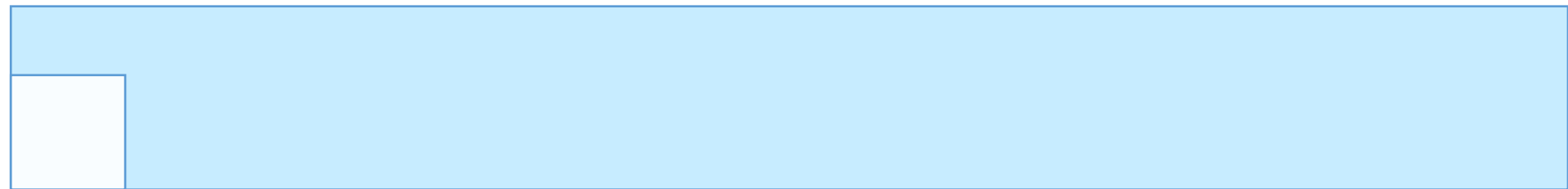


MindSpore Transformers Mcore模型迁移流程
通过封装GPTModel基类进行模型搭建



下载HF开源模型

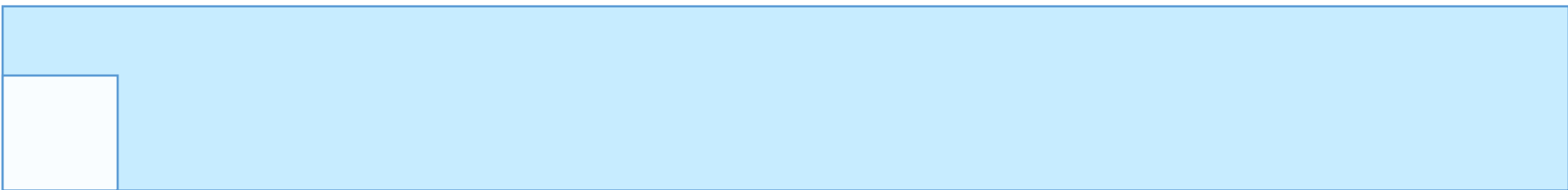
Hugging Face



Hugging Face 官网是官方推荐的模型下载渠道，支持使用 transformers 和 huggingface_hub 等工具进行模型下载。

<https://huggingface.co>

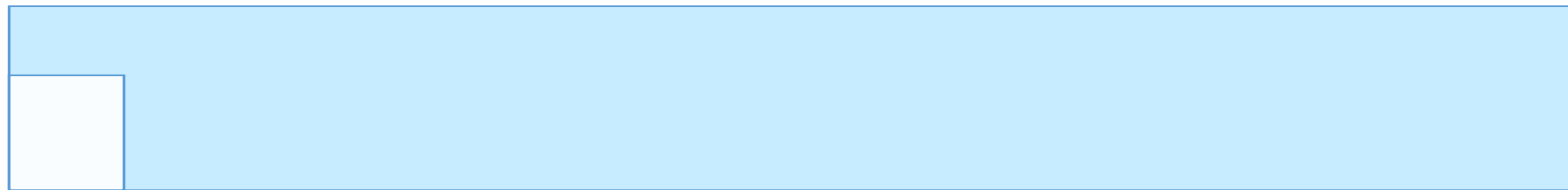
ModelScope



ModelScope（魔搭社区）是一个模型开源社区，提供部分 Hugging Face 模型资源，适用于国内用户。

<https://www.modelscope.cn>

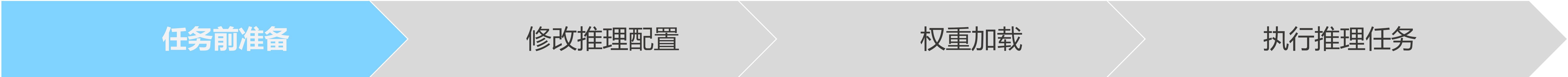
魔乐社区



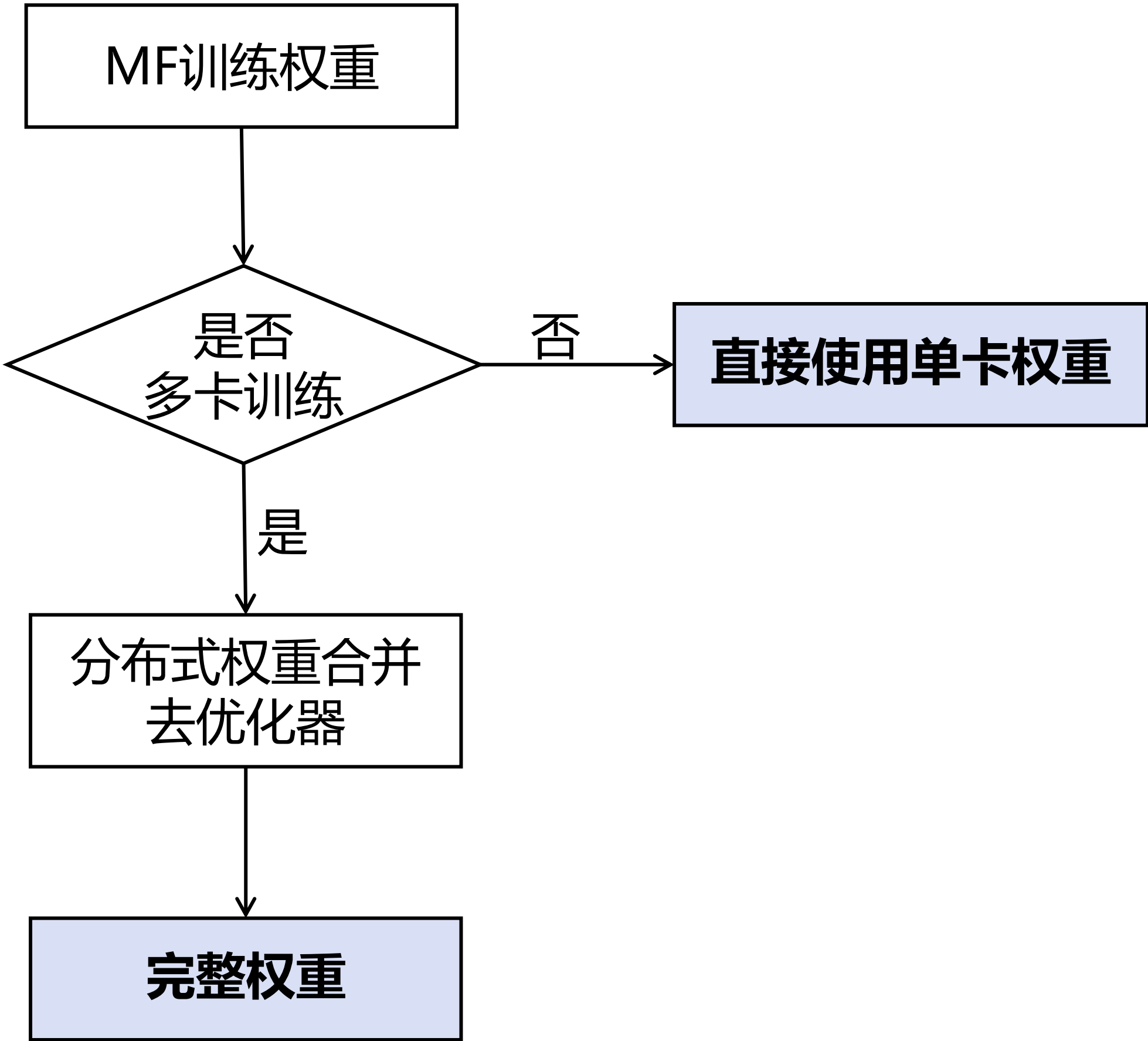
魔乐社区是一个由开发者社区维护的模型分享平台，提供部分热门 Hugging Face 模型的镜像下载，适合国内用户快速获取模型资源。

<https://modelers.cn>

直接复用HF生态 无需修改开源文件



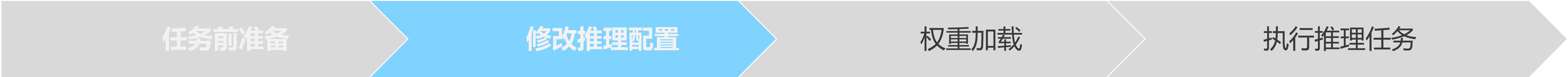
训转推模型



```
python toolkit/safetensors/unified_safetensors.py
--src_strategy_dirs output/strategy \
--mindspore_ckpt_dir output/checkpoint \
--output_dir /path/to/unified_train_ckpt \
--file_suffix "1000_1" \
--filter_out_param_prefix "adam_" \
--has_redundancy False
```

权重合并命令示例

参数名	参数说明
src_strategy_dirs	源权重对应的分布式策略文件路径
mindspore_ckpt_dir	分布式权重路径
output_dir	目标权重的保存路径
file_suffix	目标权重文件的命名后缀
filter_out_param_prefix	自定义过滤掉部分参数
has_redundancy	合并的源权重是否是冗余的权重



配置类型	类型说明	配置项	配置项说明
基础配置	通过该部分配置，能够基于当前模型拉起一个简单的推理任务	权重配置	指定模型权重格式和加载路径，用于模型初始化和权重在线加载。
		并行配置	定义模型在多设备上的分布式并行策略，包括数据、模型、专家和流水线并行的划分方式。
高级配置	通过该部分配置，可支持自定义模型结构和文本生成相关参数，控制模型输出结果	模型配置	主要用于定义模型内部计算的数据类型（如bfloat16、float32）等精度相关参数。 实际模型超参支持直接读取HF模型结构文件，也支持在YAML文件进行覆盖。
		文本生成配置	控制推理解码行为，如生成长度、采样策略（top-k、top-p、temperature）等。

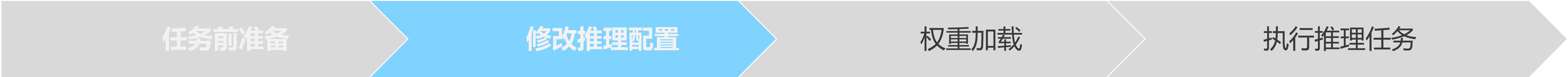
推理任务YAML模板配置

```
# 权重配置
load_ckpt_format: 'safetensors'
pretrained_model_dir: '/path/safetensors_dir'

# 并行配置
parallel_config:
  data_parallel: 1      # Number of model parallel
  model_parallel: 2     # Number of model parallel
  expert_parallel: 2    # Number of expert parallel
  pipeline_stage: 2     # Number of pipeline parallel

# 模型配置
model:
  model_config:
    compute_dtype: 'bfloat16'
    layernorm_compute_dtype: 'float32'
    rotary_dtype: 'bfloat16'
    params_dtype: 'bfloat16'

# 文本生成配置
generation_config:
  max_new_tokens: 20
  top_k: 1
  top_p: 1
  temperature: 0.6
...
```

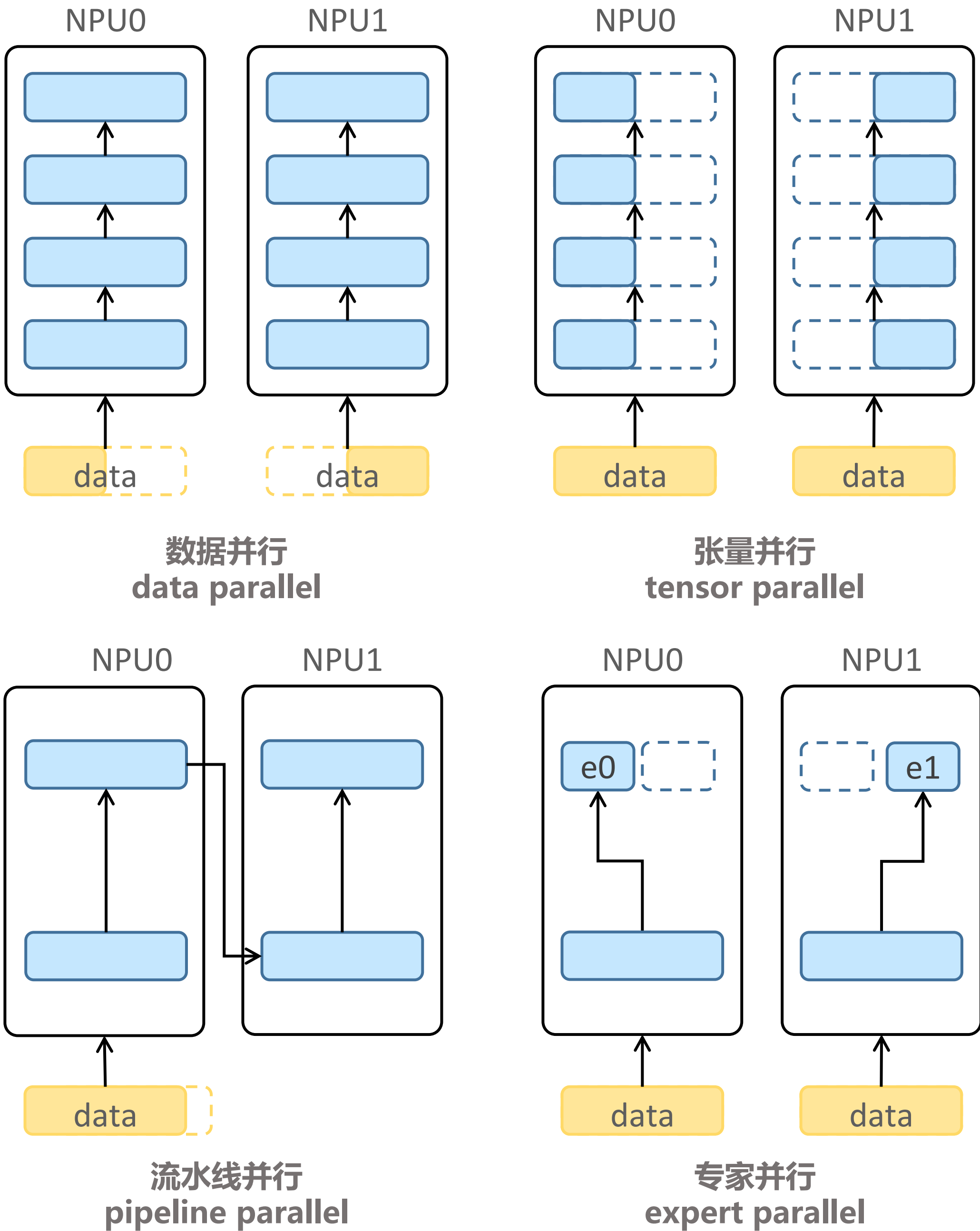
并行配置

为在有限集群资源下高效执行大规模模型推理，MindSpore Transformers 实现了多种并行策略，并支持多维混合并行，保障大模型推理的高效率与可扩展性。

```
# 并行配置
parallel_config:
  data_parallel: 1      # 数据并行
  model_parallel: 2     # 张量并行
  pipeline_stage: 2    # 流水线并行
  expert_parallel: 2    # 专家并行

parallel:
  parallel_mode: 'MANUAL_PARALLEL' #推理使用前端并行
  enable_alltoall: False
```

并行策略	基本介绍	特点	支持情况
数据并行 (DP)	每个设备保存完整的模型副本，输入数据按批次 (batch) 分片，各设备独立处理不同样本	提升吞吐量，适合多请求并发	✓
张量并行 (TP)	将模型中的权重或计算操作（如矩阵乘法）沿特定维度切分到多个设备上协同完成	显著降低每卡模型参数显存，但通信量较大	✓
流水线并行 (PP)	将模型按层数划分为多个阶段，不同设备负责不同层，输入数据像流水线一样依次通过各阶段。	减少每卡层数和显存，通信量较小，但会存在气泡	✓
专家并行 (EP)	不同的“专家网络”被分布到不同设备上，路由机制决定激活哪些专家。	MoE结构专用，能够节省每卡专家参数内存	✓
序列并行 (SP)	以输入序列长度这一维度进行分片，不同设备处理序列的不同部分，并在关键层同步中间结果。	降低长序列引起的激活显存，有效支持长文本生成	开发中



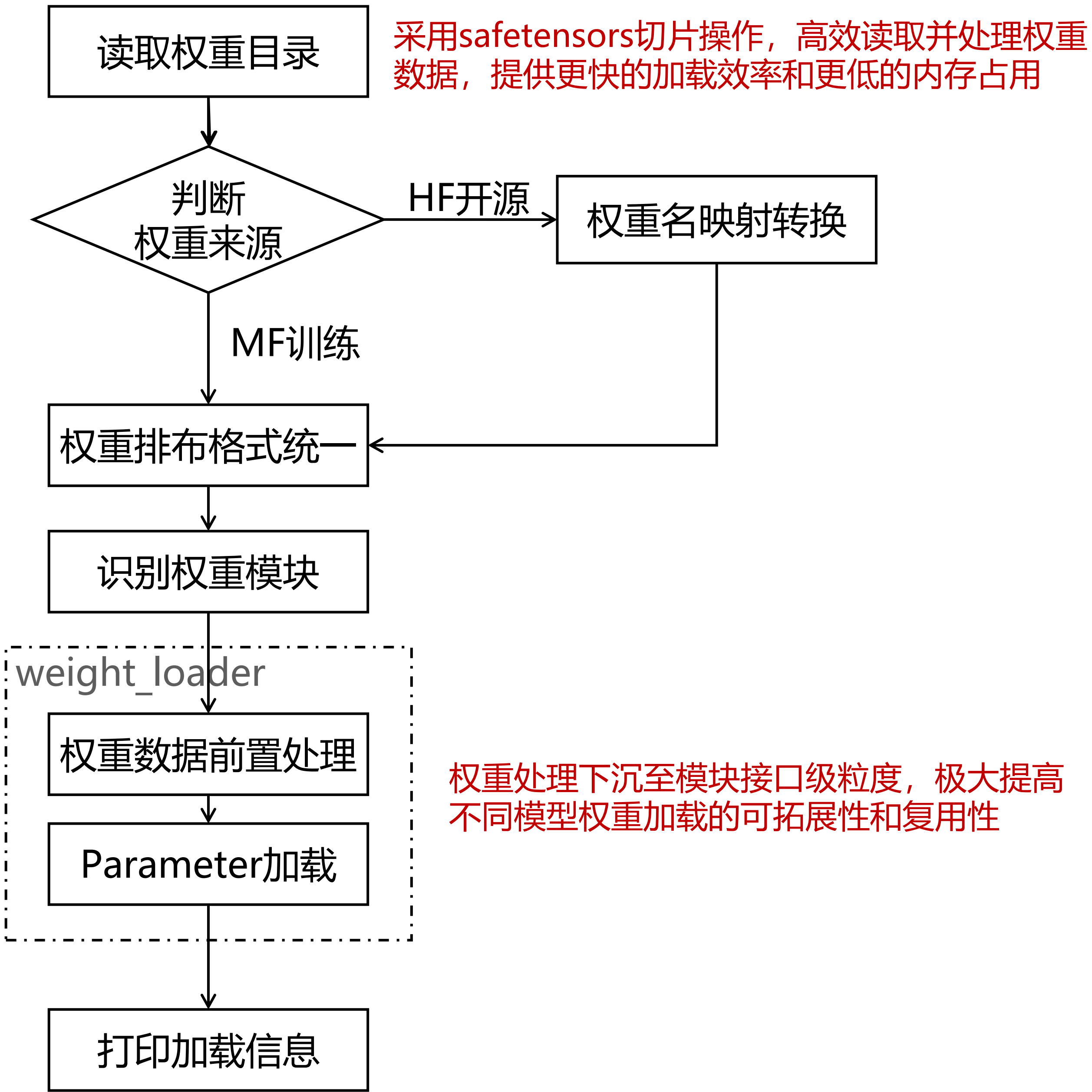
任务前准备

修改推理配置

权重加载

执行推理任务

MindSpore Transformers支持HF开源权重和训转推权重在线加载



权重名映射表

HF权重key	MF权重key
model.embed_tokens.weight	embedding.word_embeddings.weight
model.layers.{layer_number}.input_layernorm.weight	decoder.layers.{layer_number}.input_layernorm.weight
model.layers.{layer_number}.self_attn.q_proj.weight	decoder.layers.{layer_number}.self_attention.linear_qkv.weight
model.layers.{layer_number}.self_attn.k_proj.weight	
model.layers.{layer_number}.self_attn.v_proj.weight	
model.layers.{layer_number}.self_attn.o_proj.weight	decoder.layers.{layer_number}.self_attention.linear_proj.weight
model.layers.{layer_number}.mlp.down_proj.weight	decoder.layers.{layer_number}.mlp.linear_fc2.weight
model.layers.{layer_number}.mlp.gate_proj.weight	
model.layers.{layer_number}.mlp.up_proj.weight	
	decoder.layers.{layer_number}.mlp.linear_fc1.weight

ColumnParallelLinear

```
def __init__(input_size: int, ...)
def construct(input_, weight)
def weight_loader(param, load_weight, loaded_shard_id)
```

RowParallelLinear

```
def __init__(input_size: int, ...)
def construct(input_)
def weight_loader(param, load_weight)
```

ColumnParallelGroupedLinear

```
def __init__(num_local_experts: int, ...)
def construct(input_parallel, weight, ...)
def weight_loader(param, load_weight, shard_id, expert_id)
```

RowParallelGroupedLinear

```
def __init__(num_local_experts: int, ...)
def construct(input_, weight, ...)
def weight_loader(param, load_weight, shard_id, expert_id)
```


任务前准备

修改推理配置

权重加载

执行推理任务

run_mindformer.py

MindSpore Transformers提供了一键式拉起推理任务脚本run_mindformer.py，可以通过执行该脚本启动对应配置下的推理任务

```
# 使用run_mindformer.py 启动一个单卡任务
python run_mindformer.py --config {YAML_PATH} --run_mode predict --use_parallel False --predict_data '介绍一下北京故宫'
```

msrun启动

MindSpore提供的动态组网启动方式的封装，用户可使用msrun以单个命令行指令的方式在各节点拉起多进程分布式任务，并且无需手动设置动态组网环境变量，MindSpore Transformers提供了更便捷的封装脚本msrun_launcher.sh，供用户用于不同场景下的快速启动。

```
# 默认启动一个单机8卡任务
bash scripts/msrun_launcher.sh "run_mindformer.py
--config {YAML_PATH} \
--run_mode predict \
--use_parallel True \
--parallel_config.model_parallel 8 \
--predict_data '介绍一下北京故宫'"
```

单机多卡任务拉起方式

```
# 节点0，节点ip为192.168.1.1，作为主节点，总共16卡且每个节点8卡
bash scripts/msrun_launcher.sh "run_mindformer.py --config {YAML_PATH} --run_mode predict --use_parallel
True --predict_data '介绍一下北京故宫'" 16 8 192.168.1.1 8118 0 output/msrun_log False 300

# 节点1，节点ip为192.168.1.2，节点0与节点1启动命令仅参数NODE_RANK不同
bash scripts/msrun_launcher.sh "run_mindformer.py --config {YAML_PATH} --run_mode predict --use_parallel
True--predict_data '介绍一下北京故宫'" 16 8 192.168.1.1 8118 1 output/msrun_log False 300
```

多机任务拉起方式



执行结果信息

3. MindSpore Transformers量化推理能力一览

MindSpore Transformers 量化推理全流程

量化前准备

修改量化配置

执行量化校准

修改推理配置

启动量化推理

- 确定指定规格模型
- 选择量化算法

权重配置
并行配置
推理模式配置

- 构造校准数据集
- 配置量化策略
- 执行量化校准
- 保存量化模型

权重配置
并行配置
推理模式配置
量化后端配置

msrun快速启动
单机单卡 | 单机多卡

量化前准备

修改量化配置

执行量化校准

修改推理配置

启动量化推理

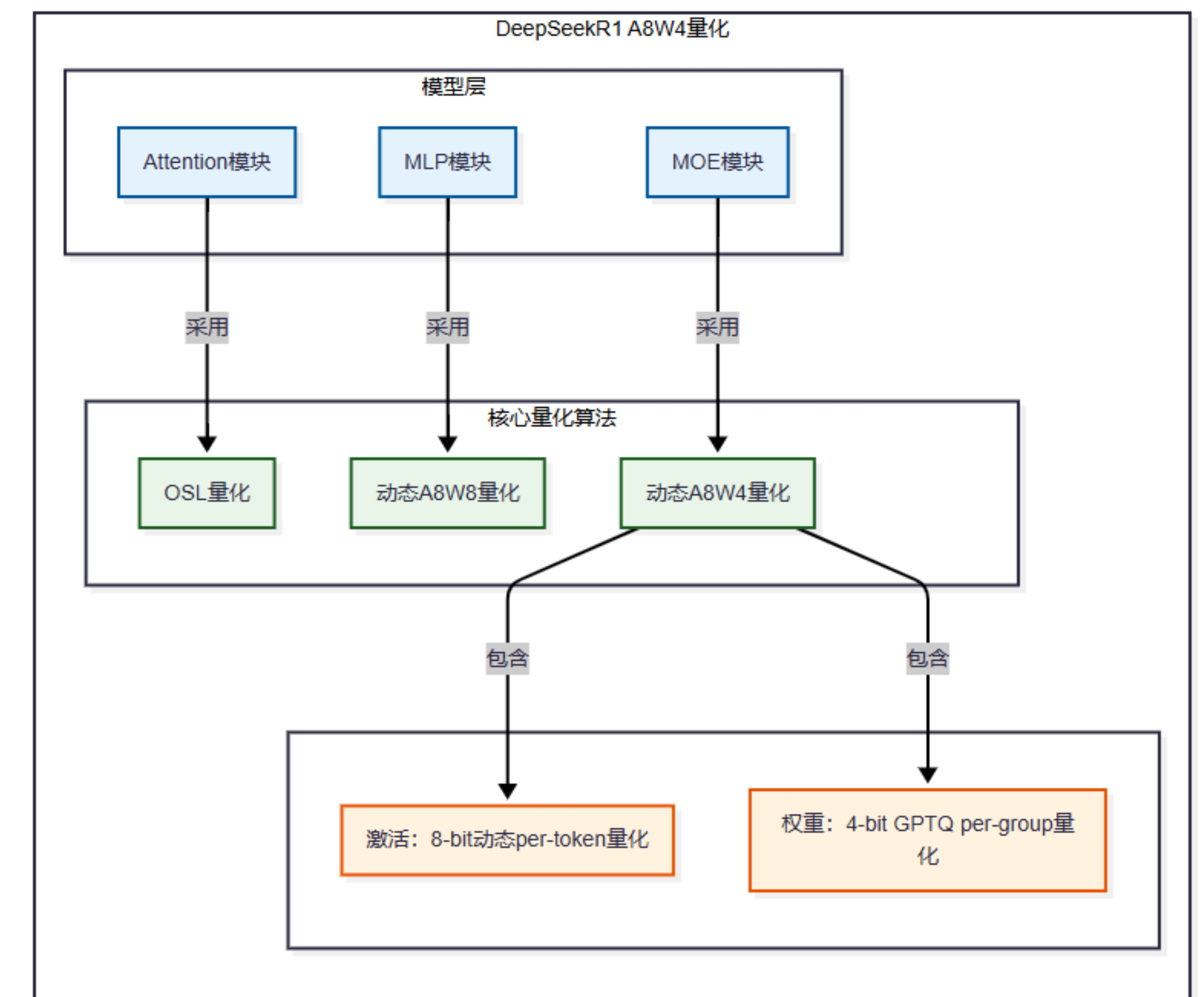
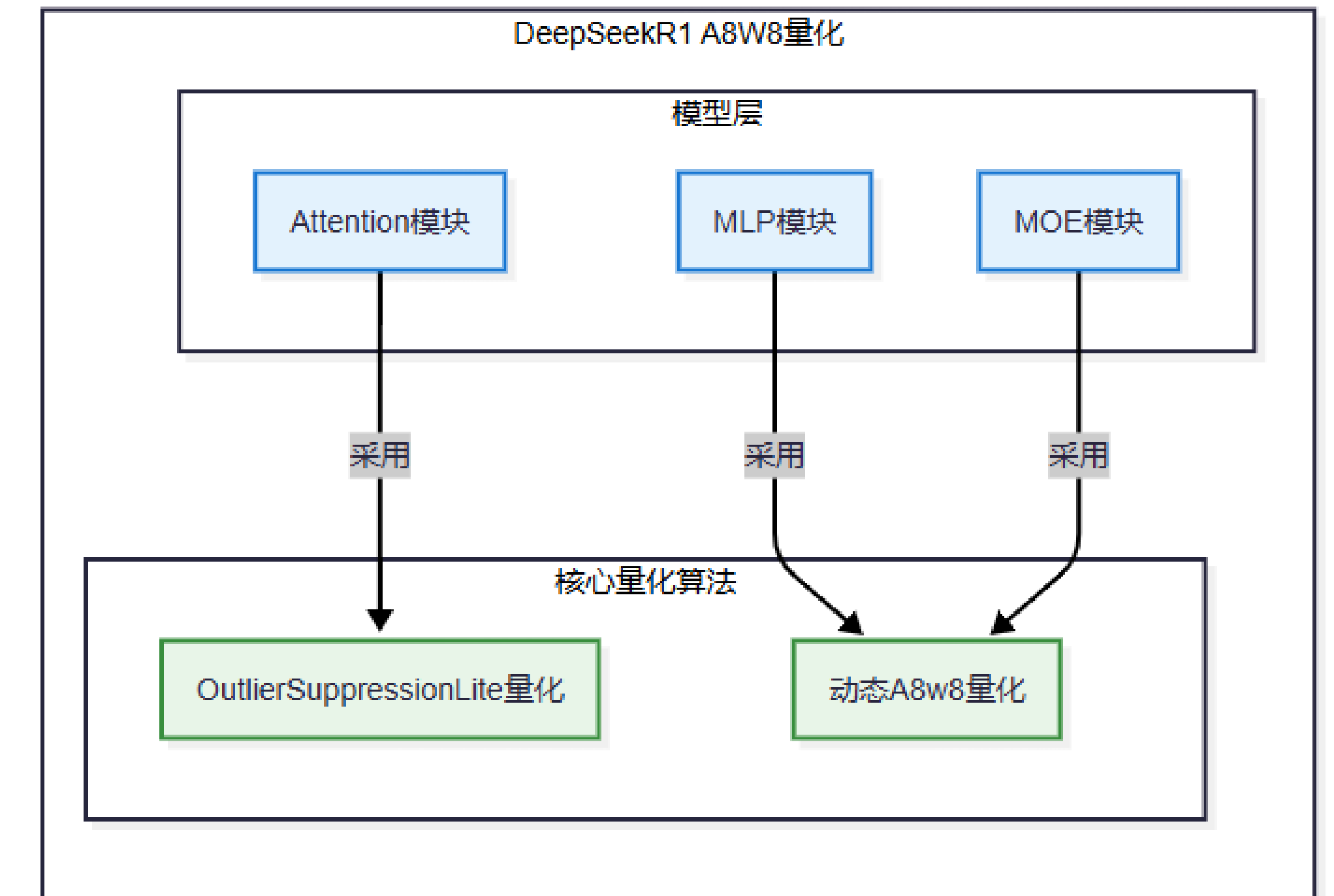
选择模型

- Released（发布级）：通过测试团队验收，评测精度与标杆满足对齐标准；
- Validated（验证级）：通过开发团队自验证，评测精度与标杆满足对齐标准；
- Preliminary（初步级）：通过开发者初步自验证，功能完整可试用，推理输出符合逻辑但精度未严格验证；
- Untested（未测试级）：功能可用但未经系统测试，精度未验证，支持用户自定义开发使能；
- Community（社区级）：社区贡献的 MindSpore 原生模型，由社区开发维护。

选择量化算法

已支持算法	简要介绍
OutlierSuppressionLite算法	由华为泰勒实验室与MindSpore团队联合开发，在SmoothQuant基础上为网络中的每个矩阵分别搜索最优超参 α 值
A8W4量化	由MindSpore团队开发的层间混合量化算法，激活使用动态per-token的8-bit量化，权重使用per-group的4-bit GPTQ量化
SmoothQuant算法	A8W8量化，通过smooth_scale将激活的量化难度转移至权重
GPTQ算法	对某个block内的所有参数逐个量化，弥补量化带来的精度损失
动态量化	激活或KVCache进行动态per-token量化
AWQ算法	通过离线网格搜索的方式实现权重低比特量化
RoundToNearest算法	朴素的后量化算法，其取整方式使用了四舍五入的方式

支持对模型不同层采取不同的量化策略，组合多种量化算法



量化前准备

修改量化配置

执行量化校准

修改推理配置

启动量化推理

use_parallel = True 多卡量化
use_parallel = False 单卡量化

model_parallel : 多卡并行量化所需的卡数
与use_parallel = True 同时开启

pretrained_model_dir :需要量化的权重文件
夹路径

mode设置为1,
使用动态图进行量化校准

Qwen3量化配置

```
seed: 0
output_dir: './output' # path to save checkpoint/strategy
load_checkpoint: ''
use_parallel: False
run_mode: 'predict'
use_legacy: False
load_ckpt_format: 'safetensors'

trainer:
  type: CausalLanguageModelingTrainer
  model_name: 'qwen3'

# default parallel of device num = 8 for Atlas 800T A2
parallel_config:
  data_parallel: 1
  model_parallel: 1

# HuggingFace file directory
pretrained_model_dir: '/path/to/qwen3-8b'
model:
  model_config:
    compute_dtype: "bfloat16"
    layernorm_compute_dtype: "float32"
    softmax_compute_dtype: "float32"
    rotary_dtype: "bfloat16"
    params_dtype: "bfloat16"

# mindspore context init config
context:
  mode: 1 #0--Graph Mode; 1--Pynative Mode
  enable_graph_kernel: False
  ascend_config:
    precision_mode: "must_keep_origin_dtype"
    max_device_memory: "59GB"
    save_graphs: False
    save_graphs_path: "./graph"

# parallel context config
parallel:
  parallel_mode: "MANUAL_PARALLEL"
  enable_alltoall: False
```


MindSpore Transformers 量化推理全流程

量化前准备

修改量化配置

执行量化校准

修改推理配置

启动量化推理

构造校准数据集

```
seq_ = 2048
max_decode_length = 1024
ignore_token_id = tokenizer.pad_token_id
ds = get_datasets(ds_type, ds_path, 'train', 1, seq_, max_decode_length, tokenizer,
                 ignore_token_id, 1, False, n_samples=n_samples)
```

量化策略配置

```
a8w8_dynamic_cfg = PTQConfig(mode=PTQMode.QUANTIZE, backend=BackendTarget.ASCEND,
                             weight_quant_dtype=msdtype.int8, act_quant_dtype=msdtype.int8,
                             act_quant_granularity=QuantGranularity.PER_TOKEN,
                             opname_blacklist=['output_layer'])
cfg = a8w8_dynamic_cfg
layer_policies = OrderedDict()
```

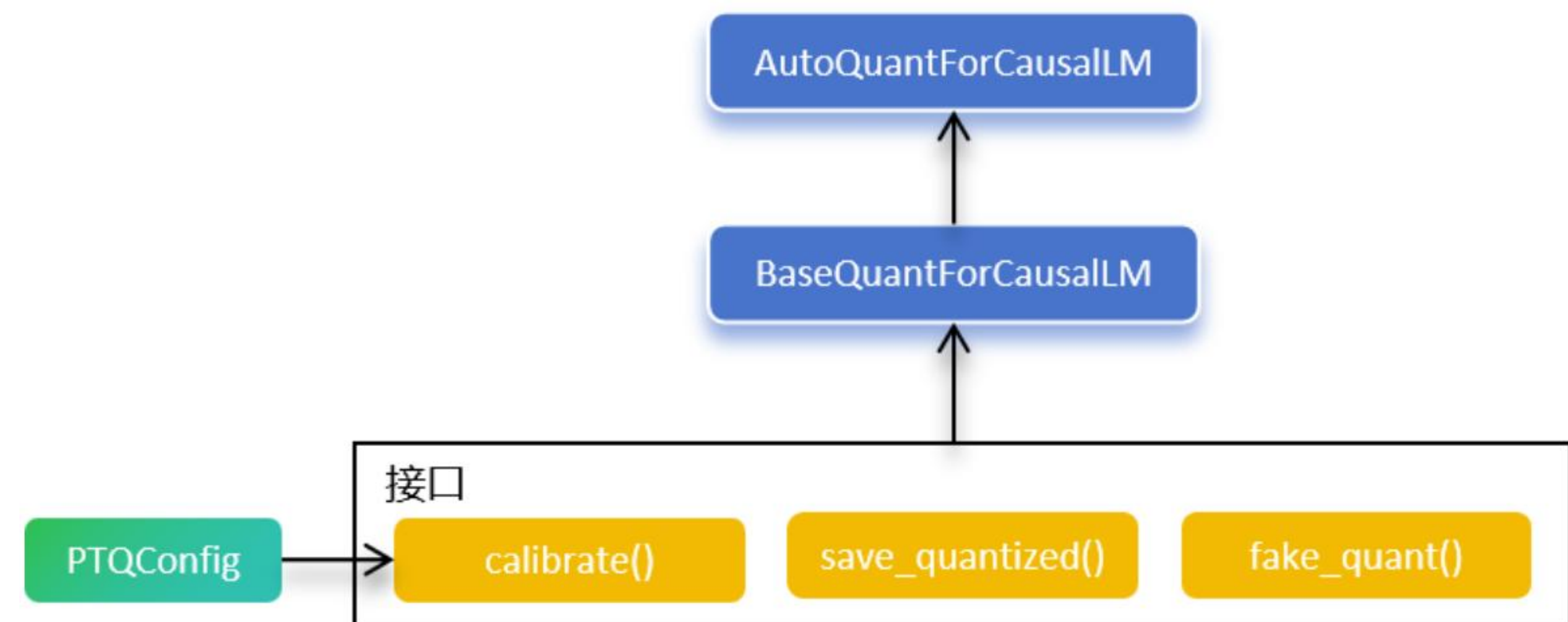
动态量化策略配置

```
a8w8_dynamic_cfg = PTQConfig(mode=PTQMode.QUANTIZE, backend=BackendTarget.ASCEND,
                             weight_quant_dtype=msdtype.int8, act_quant_dtype=msdtype.int8,
                             act_quant_granularity=QuantGranularity.PER_TOKEN,
                             opname_blacklist=['output_layer'])
smoothquant_cfg = PTQConfig(mode=PTQMode.QUANTIZE, backend=BackendTarget.ASCEND,
                             weight_quant_dtype=msdtype.int8, act_quant_dtype=msdtype.int8,
                             act_quant_granularity=QuantGranularity.PER_TENSOR,
                             outliers_suppression=OutliersSuppressionType.SMOOTH,
                             opname_blacklist=['output_layer', 'linear_fc2'])
osl_cfg = PTQConfig(mode=PTQMode.QUANTIZE, backend=BackendTarget.ASCEND,
                    weight_quant_dtype=msdtype.int8, act_quant_dtype=msdtype.int8,
                    act_quant_granularity=QuantGranularity.PER_TENSOR,
                    outliers_suppression=OutliersSuppressionType.OUTLIER_SUPPRESSION_LITE,
                    opname_blacklist=['output_layer', 'linear_fc2'])
cfg = a8w8_dynamic_cfg
layer_policies = OrderedDict({'r'.*\[0-9]\.self_attention.*': osl_cfg,
                             'r'.*\[1][0-9]\.self_attention.*': smoothquant_cfg,
                             })
```

a8w8量化策略配置

初始化网络

```
mfconfig = MindFormerConfig(config_path)
tokenizer = AutoTokenizer.from_pretrained(mfconfig.pretrained_model_dir)
model = AutoQuantForCausalLM.from_pretrained(config_path)
```



网络校准

保存量化模型

MindSpore Transformers 量化推理全流程

量化前准备

修改量化配置

执行量化校准

修改推理配置

启动量化推理

Qwen3-4B推理配置

use_parallel = True 多卡量化推理
use_parallel = False 单卡量化推理

model_parallel : 多卡并行量化推理所需的卡数
与use_parallel = True 同时开启

pretrained_model_dir :量化后的权重文件夹
路径

mode设置为0
切回静态图

```
seed: 0
output_dir: './output' # path to save checkpoint/str
load_checkpoint: ''
use_parallel: False
run_mode: 'predict'
use_legacy: False
load_ckpt_format: 'safetensors'

trainer:
  type: CausalLanguageModelingTrainer
  model_name: 'qwen3'

# default parallel of device num = 8 for Atlas 800T
parallel_config:
  data_parallel: 1
  model_parallel: 1

# HuggingFace file directory
pretrained_model_dir: '/path/to/quantized_model'
model:
  model_config:
    compute_dtype: "bfloat16"
    layernorm_compute_dtype: "float32"
    softmax_compute_dtype: "float32"
    rotary_dtype: "bfloat16"
    params_dtype: "bfloat16"

# mindspore context init config
context:
  mode: 0 #0--Graph Mode; 1--Pynative Mode
  enable_graph_kernel: False
  ascend_config:
    precision_mode: "must_keep_origin_dtype"
    max_device_memory: "59GB"
    save_graphs: False
    save_graphs_path: "./graph"

# parallel context config
parallel:
  parallel_mode: "MANUAL_PARALLEL"
  enable_alltoall: False
```

```
{
  "architectures": [
    "Qwen3ForCausalLM"
  ],
  "attention_bias": false,
  "attention_dropout": 0.0,
  "bos_token_id": 151643,
  "eos_token_id": 151645,
  "head_dim": 128,
  "hidden_act": "silu",
  "hidden_size": 4096,
  "initializer_range": 0.02,
  "intermediate_size": 12288,
  "max_position_embeddings": 40960,
  "max_window_layers": 36,
  "model_type": "qwen3",
  "num_attention_heads": 32,
  "num_hidden_layers": 36,
  "num_key_value_heads": 8,
  "rms_norm_eps": 1e-06,
  "rope_scaling": null,
  "rope_theta": 1000000,
  "sliding_window": null,
  "tie_word_embeddings": false,
  "torch_dtype": "bfloat16",
  "transformers_version": "4.51.0",
  "use_cache": true,
  "use_sliding_window": false,
  "vocab_size": 151936,
  "quantization_config": {
    "quant_method": "golden-stick"
  }
}
```

config.json中增加量化配置

量化前准备

修改量化配置

执行量化校准

修改推理配置

启动量化推理

run_mindformer.py

MindSpore Transformers提供了一键式拉起推理的任务编排脚本run_mindformer.py，可以通过执行该脚本启动对应配置下的推理任务；

```
# 使用run_mindformer.py 启动一个单卡任务
python run_mindformer.py --config {CONFIG_PATH} --run_mode predict --use_parallel False --pretrained_model_dir '/path/hf_dir' --input_data '介绍下北京故宫'
```

msrun启动

MindSpore提供的动态组网启动方式的封装，用户可使用msrun以单个命令行指令的方式在各节点拉起多进程分布式任务，并且无需手动设置动态组网环境变量，MindSpore Transformers提供了更便捷的任务封装脚本msrun_launcher.sh，供用户用于不同场景下的快速启动。

```
# 启动一个2卡任务
bash scripts/msrun_launcher.sh "run_mindformer.py \
--config configs/qwen3/predict_qwen3.yaml \
--run_mode predict \
--use_parallel True \
--pretrained_model_dir '/path/hf_dir' \
--parallel_config.data_parallel 1 \
--parallel_config.model_parallel 2 \
--predict_data '介绍下北京故宫'" 2
```


4. MindSpore Transformers推理快速上手

实操演示

MindSpore Transformers Qwen3推理



MindSpore Transformers Qwen3-4B推理任务实践.html

实操演示

MindSpore Transformers Qwen3量化推理



MindSpore Transformers Qwen3-4B量化推理任务实践.html

SIG组织

昇思社区

昇腾社区

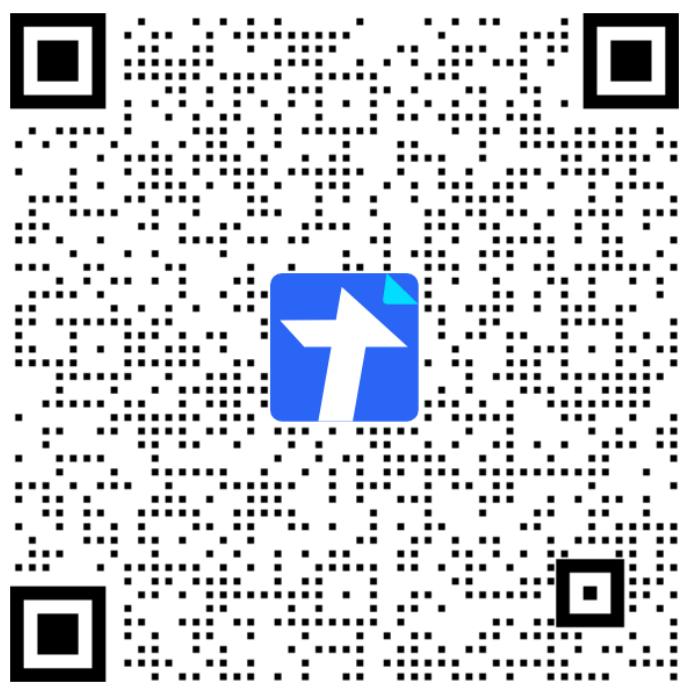
魔乐社区



课程交流群



MindSpore Transformers SIG群



MindSpore Transformers 内测文档



MindSpore Transformers 开源仓库

谢谢观看！